

Lab Report

23CSE212 – Principles of Functional Languages

Criteria	Excellent	Good	Poor
Timely Submission			
Correctness of lab assignment			
Total Marks			
Signed By Lab Instructor			

Lab Session No: 8.1

Date: 16-05-2025

Question 01:

Create a Shape type with constructors Circle Float and Rectangle Float Float. Write a function `area :: Shape -> Float` to compute area.

Test cases:

-- area (Circle 3) ==> 28.27...
-- area (Rectangle 4 5) ==> 20

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre>data Shape = Circle Float Rectangle Float Float area :: Shape -> Float area (Circle r) = pi * r * r area (Rectangle l b) = l * b</pre>	<pre>ghci> area (Circle 5) 78.539816 ghci> area (Rectangle 2 6) 6.0 ghci> :t area area :: Shape -> Float</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 02:

Define a newtype called `Celsius` that wraps a `Float`. Write a function `toFahrenheit :: Celsius -> Float`.

Test case (give 2 to 3)

-- `toFahrenheit (Celsius 0) => 32.0`

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre>newtype Celsius = Celsius Float toFahrenheit :: Celsius -> Float toFahrenheit (Celsius c) = (c * 9/5) + 32</pre>	<pre>ghci> toFahrenheit (Celsius 100) 212.0 ghci> toFahrenheit (Celsius 150) 302.0 ghci> :t toFahrenheit toFahrenheit :: Celsius -> Float</pre>

Question 03:

Create a `TrafficLight` type with values `Red`, `Yellow`, and `Green`. Write a function `nextLight :: TrafficLight -> TrafficLight` that simulates the next state.

Sample Test Case

-- `nextLight Red => Green`

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
------	----------------------------

Lab Report

23CSE212 – Principles of Functional Languages

```
data TrafficLight = Red | Yellow |  
Green deriving (Show)
```

```
nextLight :: TrafficLight ->  
TrafficLight
```

```
nextLight Red = Green  
nextLight Green = Yellow  
nextLight Yellow = Red
```

```
ghci> nextLight Red  
Green  
ghci> nextLight Yellow  
Red  
ghci> nextLight Green  
Yellow  
ghci> :t nextLight  
nextLight :: TrafficLight ->  
TrafficLight
```

Question 04:

Define a `Person` type using data with fields: `name: String`, `age: Int`. Write a function `isAdult :: Person -> Bool` that returns `True` if the person is 18 or older.

Sample Test Case

```
-- isAdult (Person "John" 20) => True
```

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code

```
data Person = Person {name ::  
String, age :: Int} deriving (Show)  
  
isAdult :: Person -> Bool  
isAdult person = age person >= 18
```

Testcases (Input & Output)

```
ghci> isAdult (Person "Jack" 12)  
False  
ghci> isAdult (Person "Emily" 22)  
True  
ghci> :t isAdult  
isAdult :: Person -> Bool
```

Question 05:

Define a data type `Grade` with constructors `A`, `B`, `C`, `D`, and `F`. Write a function `isPass :: Grade -> Bool` that returns `True` for grades `A–D`.

Lab Report

23CSE212 – Principles of Functional Languages

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code

```
data Grade = A | B | C | D | E | F
deriving (Eq, Show)
isPass :: Grade -> Bool
isPass g
    | g==F = False
    | otherwise = True
```

Testcases (Input & Output)

```
ghci> isPass A
True
ghci> isPass F
False
ghci> :t isPass
isPass :: Grade -> Bool
```

Question 06:

Create a data type `Coordinate` to represent 2D coordinates. Write a function `distanceBetween :: Coordinate -> Coordinate -> Float`.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code

```
data Coordinate = Coordinate Float
Float deriving (Show)
distanceBetween :: Coordinate ->
Coordinate -> Float
distanceBetween (Coordinate x1 y1)
(Coordinate x2 y2) =
    sqrt ((x2 - x1)^2 + (y2 - y1)^2)
```

Testcases (Input & Output)

```
ghci> distanceBetween (Coordinate
(5, 5), Coordinate (1, 1))
5.656854
ghci> distanceBetween (Coordinate
(3, 4), Coordinate (0, 0))
5.0
```

Question 07: Define a `Book` type and write a function `discountBook :: Book -> Float -> Book` that reduces the price by a given percentage.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Lab Report

23CSE212 – Principles of Functional Languages

Code	Testcases (Input & Output)
<pre>data Book = Book { title :: String, author :: String, price :: Float } deriving (Show) discountBook :: Book -> Float -> Book discountBook b percent = b { price = price b * (1 - percent / 100) }</pre>	<pre>ghci> discountBook (Book "Sapiens" "Yuval Noah Harari" 499) 10 Book {title = "Sapiens", author = "Yuval Noah Harari", price = 449.1} ghci> discountBook (Book "The Alchemist" "Paulo Coelho" 350) 15 Book {title = "The Alchemist", author = "Paulo Coelho", price = 297.5}</pre>

Question 08:

Create a data type `Vehicle` with constructors `Car String Int` (model and year) and `Bike String Int`. Write a function `vehicleAge :: Vehicle -> Int` assuming current year is 2025.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre>data Vehicle = Car String Int Bike String Int deriving (Show) vehicleAge :: Vehicle -> Int vehicleAge (Car _ year) = 2025 - year vehicleAge (Bike _ year) = 2025 - year</pre>	<pre>ghci> vehicleAge (Car "Swift" 2018) 7 ghci> vehicleAge (Bike "Yamaha RX100" 2000) 25</pre>

Question 09: Create a type alias with `Scores = [ (String, Int) ]` representing a list of student names and their scores. Write a function `topScorer :: Scores -> String` that returns the name of the student with the highest score.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation,

Lab Report

23CSE212 – Principles of Functional Languages

streams, input/output operations (IO), applicative functors, and monads

Code

```
import Data.List (maximumBy)
type Scores = [(String,Int)]
top :: Scores ->String
top s = fst ( maximumBy
  comparescores s)
  where
    comparescores ( _ , s1) ( _,s2)=
      compare s1 s2
```

Testcases (Input & Output)

```
ghci> top [("Anjali", 76),
  ("Karthik", 88)]
"Karthik"
ghci> top [("Neha", 95), ("Arjun",
  91)]
"Neha"
ghci> :t top
top :: Scores -> String
```

Question 10: Define a type LoginInfo using record syntax with fields username :: String, isAdmin :: Bool, and lastLogin :: (Int, Int, Int) representing a date (dd, mm, yyyy). Write a function isRecentAdmin :: LoginInfo -> Bool that returns True if isAdmin is True and lastLogin is in 2024 or later.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code

```
data LoginInfo = LoginInfo
  { username    :: String
  , isAdmin     :: Bool
  , lastLogin   :: (Int, Int, Int) -
  - (day, month, year)
  } deriving (Show)

isRecentAdmin :: LoginInfo -> Bool
isRecentAdmin info = isAdmin info &&
  year >= 2024
  where
    (_, _, year) = lastLogin info
```

Testcases (Input & Output)

```
ghci> let user3 = LoginInfo "bob"
True (15, 4, 2024)
ghci> isRecentAdmin user3
True

ghci> let user4 = LoginInfo
"charlie" True (20, 2, 2022)
ghci> isRecentAdmin user4
False
```

Question 11: Define a record type Book with title :: String, price :: Float, inStock :: Bool. Write a function affordableAvailableBooks :: [Book] -> [String] that returns a list of titles of books with price under 500 and in stock.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation,

Lab Report

23CSE212 – Principles of Functional Languages

streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre> data Book = Book { title :: String , price :: Float , inStock :: Bool } deriving (Show) affordableAvailableBooks :: [Book] - > [String] affordableAvailableBooks = map title . filter isAffordableAvailable where isAffordableAvailable book = price book < 500 && inStock book </pre>	<pre> ghci> let b1 = Book "Clean Code" 500 True ghci> let b2 = Book "Wings of Fire" 350 True ghci> let b3 = Book "The Power of Now" 300 False ghci> let b4 = Book "Learn You a Haskell" 600 True ghci> affordableAvailableBooks [b1, b2, b3, b4] ["Clean Code","Wings of Fire"] </pre>

Question 12: Define a data type `IntStack` using the `data` keyword. Write a function `push :: Int -> IntStack -> IntStack` that adds an element to the top of the stack. Write a function `peek :: IntStack -> Maybe Int` that returns the top element without modifying the stack. Write a function `isEmpty :: IntStack -> Bool` that returns `True` if the stack is empty. Write a `pop` function that removes the top element and returns the rest of the stack.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre> data IntStack = Stack [Int] deriving (Show) push :: Int -> IntStack -> IntStack push x (Stack xs) = Stack (x : xs) peek :: IntStack -> Maybe Int peek (Stack []) = Nothing peek (Stack (x:_)) = Just x isEmpty :: IntStack -> Bool isEmpty (Stack xs) = null xs pop :: IntStack -> IntStack pop (Stack []) = Stack [] pop (Stack (_:xs)) = Stack xs </pre>	<pre> ghci> let s0 = Stack [] ghci> let s1 = push 5 s0 ghci> let s2 = push 20 s1 ghci> let s3 = push 50 s2 ghci> s3 [50,20,5] ghci> peek s3 Just 50 ghci> isEmpty s3 False ghci> let s4 = pop s3 ghci> s4 Stack [20,5] ghci> let s5 = pop (pop s4) ghci> isEmpty s5 True </pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 13: Define a data type `StringQueue` using the `data` keyword. Write a function `enqueue :: String -> StringQueue -> StringQueue` that adds an element to the end of the queue. Write a function `dequeue :: StringQueue -> (Maybe String, StringQueue)` that removes the front element and returns it along with the rest of the queue. Write a function `queueSize :: StringQueue -> Int` that counts the number of elements in the Queue.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
<pre>data StringQueue = Queue [String] deriving (Show) enqueue :: String -> StringQueue -> StringQueue enqueue str (Queue xs) = Queue (xs ++ [str]) dequeue :: StringQueue -> (Maybe String, StringQueue) dequeue (Queue []) = (Nothing, Queue []) dequeue (Queue (x:xs)) = (Just x, Queue xs) queueSize :: StringQueue -> Int queueSize (Queue xs) = length xs</pre>	<pre>ghci> let q0 = Queue [] ghci> let q1 = enqueue "pen" q0 ghci> let q2 = enqueue "notebook" q1 ghci> let q3 = enqueue "eraser" q2 ghci> q3 Queue ["pen","notebook","eraser"] ghci> queueSize q3 3 ghci> let (front, q4) = dequeue q3 ghci> front Just "pen" ghci> q4 Queue ["notebook","eraser"] ghci> queueSize q4 2</pre>

Question 14: Define a data type `PriorityItem` with fields: `itemName :: String`, `priority :: Int`. Define a type alias `Priority Queue` as a list of `Priority Item`. Write a function `insertItem :: PriorityItem -> PriorityQueue -> PriorityQueue` that inserts a new item and maintains the queue in ascending order of priority (i.e., smallest priority comes first). Write a function `popItem :: PriorityQueue -> (Maybe PriorityItem, PriorityQueue)` that returns the item with the highest priority (i.e., head of the list) and the remaining queue.

CO3: Apply advanced Haskell concepts, including abstract data types, lazy evaluation, streams, input/output operations (IO), applicative functors, and monads

Code	Testcases (Input & Output)
------	----------------------------

Lab Report

23CSE212 – Principles of Functional Languages

```
data PriorityItem = PriorityItem
  { itemName :: String
  , priority :: Int
  } deriving (Show)

type PriorityQueue = [PriorityItem]

insertItem :: PriorityItem ->
  PriorityQueue -> PriorityQueue
insertItem item [] = [item]
insertItem item (x:xs)
  | priority item <= priority x =
    item : x : xs
  | otherwise = x : insertItem item
    xs

popItem :: PriorityQueue -> (Maybe
  PriorityItem, PriorityQueue)
popItem []      = (Nothing, [])
popItem (x:xs) = (Just x, xs)
```

```
ghci> let q0 = []

ghci> let q1 = insertItem
  (PriorityItem "email" 2) q0
ghci> let q2 = insertItem
  (PriorityItem "meeting" 4) q1
ghci> let q3 = insertItem
  (PriorityItem "report" 1) q2
ghci> q3
[PriorityItem {itemName = "report",
  priority = 1},
  PriorityItem {itemName = "email",
  priority = 2},
  PriorityItem {itemName = "meeting",
  priority = 4}]

ghci> let (top, rest) = popItem q3
ghci> top
Just (PriorityItem {itemName =
  "report", priority = 1})

ghci> rest
[PriorityItem {itemName = "email",
  priority = 2},
  PriorityItem {itemName = "meeting",
  priority = 4}]
```